

Uniwersytet DSW Ideis Kraków

Kierunek: Informatyka Stosowana

Przedmiot: Programowanie w języku Python

System algorytmicznego handlu z analizą rynku w czasie rzeczywistym

Projekt zaliczeniowy — dokumentacja techniczna i analiza wyników

Autorzy:

Piotr Sobas — nr indeksu 15841

Wojciech Szłosek — nr indeksu 15738

Karol Wojtas — nr indeksu 15902

Kraków, 2026

Spis treści

Spis treści	2
1. Streszczenie.....	4
2. Wprowadzenie	5
2.1. Cel projektu.....	5
2.2. Motywacja.....	5
2.3. Zakres projektu	5
3. Podział prac w zespole	6
4. Użyte technologie	7
4.1. Usługi zewnętrzne (API)	7
5. Architektura systemu	8
5.1. Przepływ danych w cyklu analizy	8
5.2. Mechanizmy stanu i odporności	9
6. Opis modułów	10
6.1. capital_client.py — klient API brokera.....	10
6.2. indicators.py — wskaźniki techniczne	10
6.3. news_fetcher.py — newsy i sentyment.....	10
6.4. social_monitor.py — Google News i Reddit	11
6.5. bot.py — rdzeń i logika decyzyjna	11
6.6. trade_logger.py — logowanie transakcji i kosztów	12
6.7. notifier.py — powiadomienia Telegram	12
6.8. reporter.py — raporty Excel	12
7. Kluczowe wyzwania i rozwiązania.....	13
7.1. Wygasanie sesji z brokerem.....	13
7.2. Pomiar rzeczywistych kosztów transakcyjnych (spread)	13
7.3. Wykrywanie zamknięć przez stop-loss / take-profit.....	13
7.4. Niezawodność powiadomień.....	13
7.5. Ograniczanie kosztów wywołań modelu językowego.....	13
8. Analiza wyników.....	14
8.1. Wyniki dzienne.....	14
8.2. Wyniki według instrumentu.....	14
8.3. Analiza statystyczna	15
9. Wnioski.....	17
9.1. Wnioski inżynierskie.....	17

9.2. Wnioski badawcze.....	17
9.3. Możliwe kierunki dalszego rozwoju	17
10. Podsumowanie.....	18

1. Streszczenie

Niniejszy dokument opisuje projekt zaliczeniowy polegający na zaprojektowaniu i implementacji autonomicznego systemu algorytmicznego handlu instrumentami finansowymi (kontraktami CFD) na platformie demonstracyjnej Capital.com. System został w całości napisany w języku Python i działa jako usługa pracująca w trybie ciągłym (24/7) na serwerze z systemem Linux.

Architektura systemu opiera się na podziale na osiem niezależnych modułów, z których każdy odpowiada za jeden obszar funkcjonalny: komunikację z API brokera, obliczanie wskaźników technicznych, pobieranie newsów i sentymentu rynkowego, podejmowanie decyzji z wykorzystaniem modelu językowego (LLM), wysyłanie powiadomień, logowanie transakcji oraz generowanie raportów. Cykl analityczny uruchamiany jest co 15 minut, a decyzje handlowe (kupno, sprzedaż lub wstrzymanie) podejmowane są na podstawie połączenia analizy technicznej, danych fundamentalnych i oceny modelu językowego.

Kluczowym elementem projektu jest **uczciwa analiza skuteczności** zaimplementowanej strategii w oparciu o realnie zebrane dane (58 transakcji w okresie 15–20 maja 2026). Analiza wykazała, że strategia w obecnej formie **nie generuje statystycznie istotnej przewagi nad rynkiem** — wynik teoretyczny (po wyeliminowaniu kosztów transakcyjnych) wyniósł $-0,82$ USD, a współczynnik typu Sharpe’a był bliski zeru. Ten wniosek, choć negatywny z punktu widzenia rentowności, stanowi wartościowy rezultat naukowy i potwierdza poprawność przyjętej metodologii pomiarowej.

2. Wprowadzenie

2.1. Cel projektu

Celem projektu było zaprojektowanie i zbudowanie kompletnego, działającego systemu informatycznego, który w sposób w pełni autonomiczny analizuje sytuację na rynkach finansowych i podejmuje decyzje handlowe. Projekt miał charakter zarówno inżynierski (budowa wielomodułowej aplikacji integrującej liczne zewnętrzne usługi), jak i badawczy (weryfikacja hipotezy o skuteczności przyjętej strategii handlowej).

2.2. Motywacja

Rynki finansowe stanowią atrakcyjny obszar do nauki programowania, ponieważ łączą wiele zagadnień technicznych: pracę z zewnętrznymi interfejsami API, przetwarzanie danych w czasie rzeczywistym, obliczenia numeryczne, automatyzację oraz analizę statystyczną. Projekt umożliwił praktyczne zastosowanie tych umiejętności w jednym spójnym systemie.

2.3. Zakres projektu

System obejmuje pełny cykl życia transakcji algorytmicznej:

- pobieranie danych rynkowych (świece 15-minutowe, aktualne ceny) z API brokera,
- obliczanie wskaźników analizy technicznej (RSI, MACD, wstęgi Bollingera, średnie EMA, ATR),
- pozyskiwanie kontekstu fundamentalnego (newsy finansowe, sentyment z mediów społecznościowych),
- podejmowanie decyzji przez model językowy na podstawie zebranych danych,
- filtrowanie decyzji (poziom pewności, zgodność z trendem, okresy karencji),
- otwieranie i monitorowanie pozycji oraz wykrywanie ich zamknięcia,
- logowanie transakcji wraz z kosztami (spread), generowanie raportów Excel i wysyłanie powiadomień Telegram.

3. Podział prac w zespole

Projekt został zrealizowany przez trzyosobowy zespół. Prace podzielono na trzy obszary o zbliżonym nakładzie, tak aby każdy z autorów odpowiadał za spójny, samodzielny fragment systemu wraz z jego dokumentacją i testami.

Autor	Obszar odpowiedzialności	Moduły / zadania
Piotr Sobas	Integracja z brokerem i dane rynkowe	capital_client.py, indicators.py — komunikacja z API Capital.com, zarządzanie sesją, obliczanie wskaźników technicznych
Wojciech Szłosek	Logika decyzyjna i rdzeń systemu	bot.py, trade_logger.py — główna pętla, scheduler, integracja z modelem językowym, logika filtrów, logowanie transakcji
Karol Wojtas	Źródła kontekstu, raportowanie i powiadomienia	news_fetcher.py, social_monitor.py, notifier.py, reporter.py — newsy, sentyment, alerty Telegram, raporty Excel

Tabela 1. Podział obowiązków w zespole projektowym.

Mimo podziału na obszary, integracja modułów oraz analiza końcowa wyników były pracą wspólną całego zespołu.

4. Użyte technologie

System został zbudowany w języku Python 3 i wykorzystuje szereg bibliotek zewnętrznych oraz usług dostępnych przez API. Poniższa tabela przedstawia kluczowe zależności wraz z ich rolą w projekcie.

Technologia / biblioteka	Wersja	Zastosowanie w projekcie
requests	2.33.1	Komunikacja HTTP z API Capital.com, Telegram i Alpha Vantage
openai	2.34.0	Wywołania modelu językowego GPT-4o-mini (silnik decyzyjny)
pandas	2.3.3	Przetwarzanie szeregów czasowych przy obliczaniu wskaźników
numpy	2.2.6	Obliczenia numeryczne wspierające analizę techniczną
openpyxl	3.1.5	Generowanie raportów dziennych w formacie Excel (.xlsx)
schedule	1.2.2	Harmonogramowanie zadań (cykl co 15 min, raport dzienny)
python-dotenv	1.2.2	Wczytywanie konfiguracji i kluczy API z pliku .env
pyotp	2.9.0	Generowanie kodów TOTP do uwierzytelniania dwuskładnikowego
finnhub-python	—	Pobieranie newsów finansowych z serwisu Finnhub
feedparser	6.0.12	Parsowanie kanałów RSS (Google News)
praw	7.8.1	Pobieranie postów z serwisu Reddit (analiza sentymentu)

Tabela 2. Wykorzystane biblioteki zewnętrzne i ich rola.

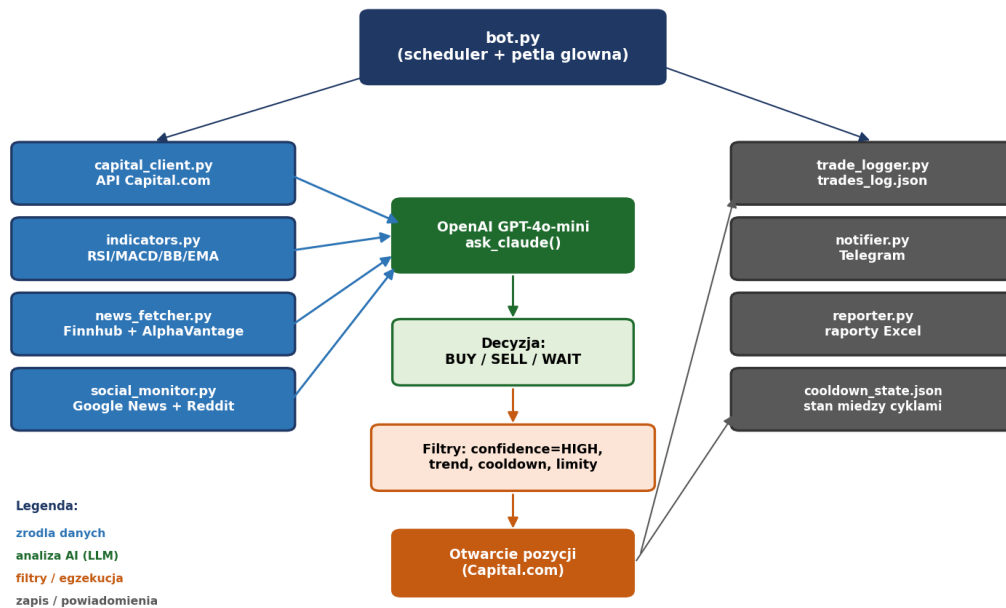
4.1. Usługi zewnętrzne (API)

- **Capital.com (demo)** — broker dostarczający dane rynkowe oraz umożliwiający otwieranie pozycji CFD na koncie demonstracyjnym.
- **OpenAI** — model językowy GPT-4o-mini pełniący rolę komponentu podejmującego decyzję handlową na podstawie dostarczonego kontekstu.
- **Finnhub oraz Alpha Vantage** — źródła newsów finansowych i gotowego wskaźnika sentymentu rynkowego.
- **Google News (RSS) oraz Reddit** — dodatkowe źródła kontekstu informacyjnego i nastrojów inwestorów.
- **Telegram Bot API** — kanał powiadomień o otwarciu/zamknięciu pozycji oraz raportów dziennych.

5. Architektura systemu

System zaprojektowano w sposób modułowy — każdy plik źródłowy odpowiada za jeden, jasno wydzielony obszar funkcjonalny. Dzięki temu poszczególne komponenty można rozwijać i testować niezależnie. Centralnym elementem jest moduł `bot.py`, który pełni rolę orkiestratora: harmonogramuje cykle analizy, zbiera dane ze źródeł, wywołuje silnik decyzyjny, stosuje filtry bezpieczeństwa i zleca otwieranie pozycji.

Architektura systemu — przepływ danych w cyklu analizy



Rysunek 1. Schemat architektury systemu i przepływu danych w pojedynczym cyklu analizy.

5.1. Przepływ danych w cyklu analizy

Pojedynczy cykl analityczny (uruchamiany co 15 minut) przebiega według następujących kroków:

1. Sprawdzenie godzin handlu i odświeżenie sesji z brokerem (keep-alive).
2. Wykrycie zamknięć istniejących pozycji (przez porównanie ze stanem poprzednim) i ich zalogowanie.
3. Dla każdego instrumentu: pobranie danych rynkowych, wskaźników technicznych oraz kontekstu newsowego i społecznościowego.
4. Wstępny filtr techniczny — odrzucenie sytuacji bez wyraźnego sygnału (oszczędność wywołań modelu).
5. Zapytanie modelu językowego o decyzję (BUY / SELL / WAIT) wraz z poziomami stop-loss i take-profit.
6. Zastosowanie filtrów: wymóg wysokiej pewności, zgodność z trendem, okres karencji, limit liczby pozycji i kapitału.

7. Otwarcie pozycji (jeśli wszystkie filtry przeszły) oraz jej zalogowanie i wysłanie powiadomienia.

5.2. Mechanizmy stanu i odporności

System zawiera mechanizmy zapewniające ciągłość działania między cyklami oraz po ewentualnym restarcie:

- **Stan karencji** (`cooldown_state.json`) — zapamiętuje czas ostatniego zamknięcia każdego instrumentu, aby uniknąć natychmiastowego ponownego wejścia w rynek; przeżywa restart usługi.
- **Rejestr znanych pozycji** (`KNOWN_POSITIONS`) — pozwala wykryć zamknięcie pozycji przez mechanizm stop-loss/take-profit po stronie brokera.
- **Automatyczne wznowianie sesji** — wygaśnięcie tokenów sesji jest wykrywane i obsługiwane przez ponowne logowanie.

6. Opis modułów

Poniżej omówiono każdy z ośmiu modułów systemu wraz z ilustrującym fragmentem kodu. Pełny kod źródłowy dostępny jest w dołączonym archiwum / repozytorium projektu.

6.1. `capital_client.py` — klient API brokera

Moduł odpowiada za całą komunikację z platformą Capital.com: logowanie (z uwierzytelnianiem dwuskładnikowym TOTP), pobieranie świec i cen, otwieranie oraz zamykanie pozycji, a także podtrzymywanie sesji. Tokeny sesji (CST oraz X-SECURITY-TOKEN) przechowywane są w obiekcie klienta i dołączane do każdego żądania.

Przykład — logowanie i pozyskanie tokenów sesji:

```
def login(self):
    """Logowanie i pobranie tokenów sesji"""
    r = requests.post(
        f"{BASE_URL}/session",
        headers={"X-CAP-API-KEY": self.api_key,
                 "Content-Type": "application/json"},
        json={"identifier": self.email,
              "password": self.password,
              "encryptedPassword": False})
    r.raise_for_status()
    self.cst = r.headers.get("CST")
    self.sec_token = r.headers.get("X-SECURITY-TOKEN")
```

Metoda `close_position` implementuje strategię awaryjną: najpierw próbuje oficjalnego zamknięcia (DELETE), a w razie niepowodzenia otwiera pozycję przeciwną o tym samym rozmiarze — co w praktyce neutralizuje ekspozycję.

6.2. `indicators.py` — wskaźniki techniczne

Moduł oblicza zestaw popularnych wskaźników analizy technicznej na podstawie listy świec. Wykorzystuje bibliotekę `pandas` do operacji na szeregach czasowych. Obliczane są: RSI (14), MACD wraz z linią sygnału i histogramem, wstęgi Bollingera, średnie wykładnicze EMA9 i EMA21 oraz wskaźnik zmienności ATR.

Przykład — obliczenie wskaźnika RSI:

```
def _rsi(series, period=14):
    delta = series.diff()
    gain = delta.where(delta > 0, 0.0)
    loss = -delta.where(delta < 0, 0.0)
    avg_gain = gain.ewm(com=period-1, min_periods=period).mean()
    avg_loss = loss.ewm(com=period-1, min_periods=period).mean()
    rs = avg_gain / avg_loss
    return 100 - (100 / (1 + rs))
```

Na podstawie obliczonych wartości moduł generuje czytelne sygnały tekstowe (np. „RSI=28 OVERSOLD → BUY”), które następnie trafiają do modelu językowego.

6.3. `news_fetcher.py` — newsy i sentyment

Moduł pozyskuje kontekst fundamentalny z dwóch źródeł: Finnhub (nagłówki newsów) oraz Alpha Vantage (gotowy wskaźnik sentymentu z oceną liczbową). Każdy instrument ma przypisany zestaw słów

kluczowych, według których filtrowane są artykuły. Wyniki są buforowane przez 30 minut, co ogranicza liczbę zapytań do płatnych API.

Przykład — mechanizm pamięci podręcznej (cache) z czasem życia 30 minut:

```
def get_news_context(symbol: str) -> dict:
    cached = _NEWS_CACHE.get(symbol)
    if cached and (_time.time() - cached[0]) < NEWS_TTL:
        return cached[1]
    news = get_finnhub_news(symbol)
    sentiment = get_alphavantage_sentiment(symbol)
    result = {...}
    _NEWS_CACHE[symbol] = (_time.time(), result)
    return result
```

6.4. social_monitor.py — Google News i Reddit

Moduł rozszerza kontekst informacyjny o newsy z kanałów RSS Google News oraz gorące posty z tematycznych subredditów (np. r/investing, r/stocks). Zaimplementowano w nim prostą analizę sentymentu opartą na zliczaniu słów o wydźwięku pozytywnym i negatywnym, zwracającą ocenę: bullish, bearish lub neutral.

Przykład — słownikowa analiza sentymentu:

```
def analyze_sentiment(texts: list) -> dict:
    bullish_words = ["surge", "rally", "gains", "bullish", "strong", ...]
    bearish_words = ["fall", "drop", "crash", "bearish", "weak", ...]
    score = 0
    for text in texts:
        t1 = text.lower()
        score += sum(1 for w in bullish_words if w in t1)
        score -= sum(1 for w in bearish_words if w in t1)
    sentiment = "bullish" if score > 2 else \
        "bearish" if score < -2 else "neutral"
    return {"sentiment": sentiment, "score": score}
```

6.5. bot.py — rdzeń i logika decyzyjna

Najważniejszy i najobszerniejszy moduł systemu. Zawiera główną pętlę, harmonogram zadań, integrację z modelem językowym oraz całą logikę filtrowania decyzji. To tutaj zbiegają się dane ze wszystkich pozostałych modułów.

Przykład — filtr decyzji: pozycja otwierana jest wyłącznie przy wysokiej pewności i zgodności z trendem:

```
if action in ("BUY", "SELL") and confidence == "HIGH":
    trend = get_trend(data['candles'])
    if action == "BUY" and trend == "DOWN":
        logging.info(f"{symbol}: BUY zablokowany – trend DOWN")
        continue
    if action == "SELL" and trend == "UP":
        logging.info(f"{symbol}: SELL zablokowany – trend UP")
        continue
    size = POSITION_SIZE.get(symbol, 1.0)
    result = capital.open_position(epic=symbol, direction=action,
                                  size=size, stop_loss=stop_loss, take_profit=take_profit)
```

Warto odnotować, że funkcja podejmująca decyzję nosi historyczną nazwę `ask_claude`, lecz w finalnej wersji wykorzystuje model **GPT-4o-mini** firmy OpenAI. Model otrzymuje ustrukturyzowany prompt zawierający dane rynkowe, wskaźniki i newsy, a odpowiada wyłącznie w formacie JSON, co ułatwia automatyczne przetwarzanie odpowiedzi.

Przykład — wymuszenie odpowiedzi w formacie JSON:

```
response = openai_client.chat.completions.create(
    model="gpt-4o-mini",
    max_tokens=300,
    messages=[{"role": "user", "content": prompt}],
    response_format={"type": "json_object"})
text = response.choices[0].message.content.strip()
return json.loads(text)
```

6.6. trade_logger.py — logowanie transakcji i kosztów

Moduł zapisuje każdą transakcję do pliku `trades_log.json`, rejestrując ceny kupna/sprzedaży (`bid/ask`) w chwili otwarcia, obliczając koszt spreadu oraz — po zamknięciu — rzeczywisty i teoretyczny wynik (`P&L`). Dostęp do pliku zabezpieczono blokadą (`Lock`) ze względu na wielowątkowość. Mechanizm „wpisu retroaktywnego” pozwala uwzględnić również pozycje, których momentu otwarcia nie udało się zarejestrować.

Przykład — obliczenie kosztu spreadu przy otwarciu pozycji:

```
spread = abs(ask - bid) if (bid and ask) else 0
trades[deal_id] = {
    "symbol": symbol, "direction": direction, "size": size,
    "open_bid": bid, "open_ask": ask,
    "spread_open": round(spread, 5),
    "spread_cost": round(spread * size, 2),
    "status": "open"}
```

Rozróżnienie wyniku **rzeczywistego** (gross `P&L`) i **teoretycznego** (bez kosztu spreadu) jest kluczowe dla analizy w rozdziale 8 — pozwala oddzielić jakość samego sygnału od wpływu kosztów transakcyjnych.

6.7. notifier.py — powiadomienia Telegram

Moduł wysyła powiadomienia na czat Telegram: o otwarciu i zamknięciu pozycji, raport dzienny oraz okresowy „sygnał życia” (`heartbeat`) potwierdzający, że system działa. Zawiera również funkcję wysyłania plików (raportów Excel) jako załączników.

6.8. reporter.py — raporty Excel

Moduł generuje sformatowany raport dzienny w formacie `.xlsx`, składający się z trzech arkuszy: „Dziennik” (wszystkie decyzje i ich uzasadnienia), „Podsumowanie” (statystyki dnia, w tym koszty spreadów i wynik teoretyczny) oraz „Spready” (szczegółowa analiza kosztów transakcyjnych w rozbiciu na instrumenty). Dane pozyskiwane są zarówno z pliku logów, jak i z modułu `trade_logger`.

7. Kluczowe wyzwania i rozwiązania

W trakcie realizacji projektu napotkano szereg problemów technicznych typowych dla systemów pracujących w czasie rzeczywistym i integrujących wiele zewnętrznych usług. Poniżej opisano najważniejsze z nich wraz z przyjętymi rozwiązaniami.

7.1. Wygasanie sesji z brokerem

Tokeny sesji Capital.com wygasają po krótkim czasie (obserwowano wygaśnięcia co ok. 10–15 minut). Bez obsługi tego zjawiska żądania API kończyłyby się błędami. **Rozwiązanie:** metoda `keep_alive` wykrywa kody odpowiedzi 401/403 i automatycznie wykonuje ponowne logowanie, dzięki czemu system działa nieprzerwanie.

7.2. Pomiar rzeczywistych kosztów transakcyjnych (spread)

W modelu CFD koszt spreadu nie jest naliczany jako osobna opłata, lecz jest „ukryty” w cenie otwarcia pozycji — co utrudnia jego pomiar. **Rozwiązanie:** zaimplementowano moduł `trade_logger`, który w momencie otwarcia zapisuje ceny bid i ask, oblicza spread i jego koszt, a następnie — po zamknięciu — wylicza wynik teoretyczny (bez spreadu). Pozwoliło to precyzyjnie określić, jaka część wyniku wynika z jakości strategii, a jaka z kosztów.

7.3. Wykrywanie zamknięć przez stop-loss / take-profit

Pozycje bywają zamykane automatycznie przez brokera (po osiągnięciu poziomu SL lub TP), bez udziału systemu. **Rozwiązanie:** w każdym cyklu system porównuje listę aktualnych pozycji z zapamiętanym stanem (`KNOWN_POSITIONS`); zniknięcie pozycji interpretowane jest jako jej zamknięcie, co uruchamia logowanie wyniku i wysłanie powiadomienia.

7.4. Niezawodność powiadomień

Zaobserwowano sporadyczne gubienie powiadomień o otwarciu pozycji (np. wskutek chwilowych problemów sieciowych). **Rozwiązanie:** dodano logowanie powodzenia/niepowodzenia wysyłki oraz mechanizm ponawiania próby po 2 sekundach, co wyeliminowało problem.

7.5. Ograniczanie kosztów wywołań modelu językowego

Każde zapytanie do modelu językowego generuje koszt. Odpytywanie modelu w każdym cyklu dla każdego instrumentu byłoby nieefektywne. **Rozwiązanie:** wprowadzono wstępny filtr techniczny — model jest pytany tylko wtedy, gdy wskaźniki sygnalizują wyraźną sytuację rynkową. Dodatkowo newsy są buforowane przez 30 minut.

8. Analiza wyników

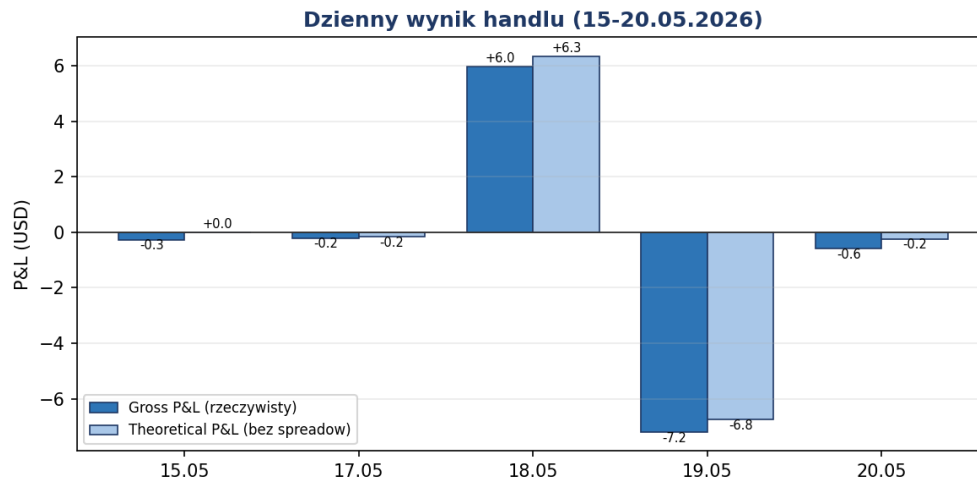
System uruchomiono na koncie demonstracyjnym i zebrano dane z 58 zamkniętych transakcji w okresie od 15 do 20 maja 2026. Poniższa analiza opiera się wyłącznie na rzeczywiście zarejestrowanych danych z pliku trades_log.json.

8.1. Wyniki dzienne

Wyniki w poszczególnych dniach charakteryzowały się bardzo wysoką zmiennością. Dwa dni (18 i 19 maja) niemal wzajemnie się zniosły: zysk teoretyczny +6,35 USD jednego dnia został zniwelowany stratą –6,76 USD następnego.

Data	Liczba transakcji	Win rate	Wynik rzeczywisty	Wynik teoretyczny	Spread
15.05	13	31%	–0,27 \$	0,00 \$	0,27 \$
17.05	2	50%	–0,21 \$	–0,17 \$	0,04 \$
18.05	13	62%	+5,98 \$	+6,35 \$	0,37 \$
19.05	16	44%	–7,21 \$	–6,76 \$	0,45 \$
20.05	14	36%	–0,59 \$	–0,24 \$	0,35 \$
RAZEM	58	43,1%	–2,30 \$	–0,82 \$	1,48 \$

Tabela 3. Zestawienie wyników dziennych.



Rysunek 2. Porównanie wyniku rzeczywistego i teoretycznego w ujęciu dziennym.

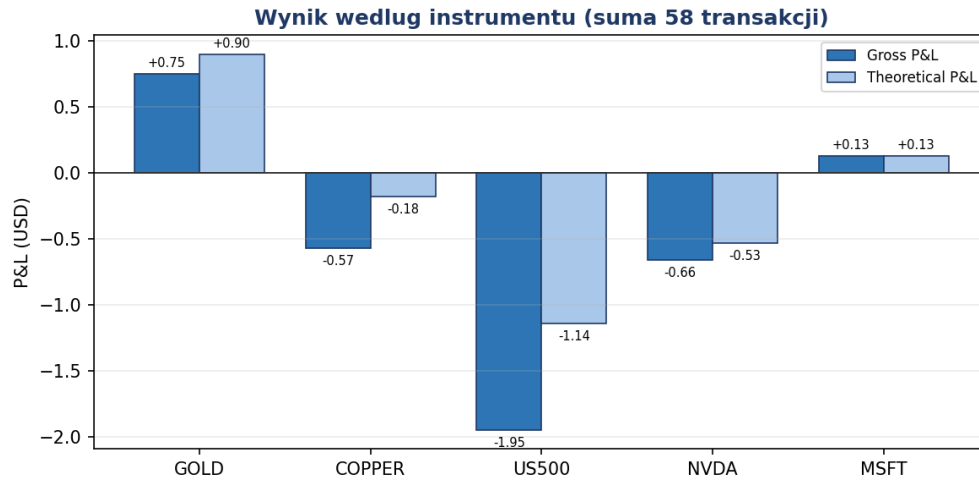
8.2. Wyniki według instrumentu

Analiza w rozbiciu na instrumenty pokazuje, że żaden z nich nie wykazał wyraźnej, powtarzalnej przewagi. Jedynie GOLD i MSFT osiągnęły nieznacznie dodatni wynik teoretyczny, jednak przy małej liczbie transakcji wynik ten mieści się w granicach przypadkowości.

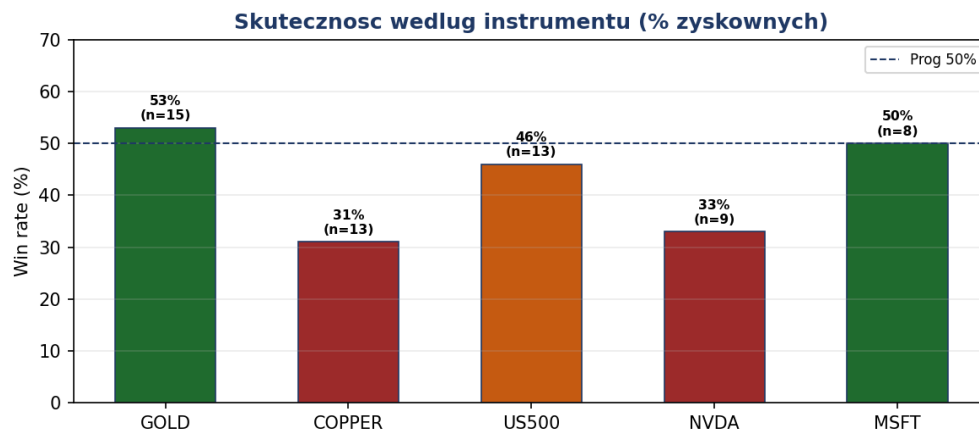
Instrument	Transakcje	Win rate	Wynik rzeczywisty	Wynik teoretyczny	Koszt spreadu
GOLD	15	53%	+0,75 \$	+0,90 \$	0,15 \$
COPPER	13	31%	–0,57 \$	–0,18 \$	0,39 \$

Instrument	Transakcje	Win rate	Wynik rzeczywisty	Wynik teoretyczny	Koszt spreadu
US500	13	46%	-1,95 \$	-1,14 \$	0,81 \$
NVDA	9	33%	-0,66 \$	-0,53 \$	0,13 \$
MSFT	8	50%	+0,13 \$	+0,13 \$	0,00 \$

Tabela 4. Wyniki w rozbiciu na instrumenty.



Rysunek 3. Wynik rzeczywisty i teoretyczny według instrumentu.



Rysunek 4. Skuteczność (win rate) według instrumentu na tle progu 50%.

8.3. Analiza statystyczna

Najważniejszym wskaźnikiem oceny strategii jest wynik teoretyczny — czyli rezultat po wyeliminowaniu kosztów spreadu, odzwierciedlający jakość samego sygnału. Wynik ten wyniósł -0,82 USD na 58 transakcji, czyli średnio -0,014 USD na transakcję.

Metryka	Wartość	Interpretacja
Liczba transakcji	58	Próba o ograniczonej licznosci
Win rate (ogólny)	43,1%	25 zyskowych / 33 stratne

Metryka	Wartość	Interpretacja
Wynik rzeczywisty	-2,30 \$	Po uwzględnieniu spreadów
Wynik teoretyczny	-0,82 \$	Bez spreadów — jakość sygnału
Średnia / transakcję	-0,040 \$	Bliska zeru
Odchylenie standardowe	0,898 \$	Wysoka zmienność wyników
Wskaźnik typu Sharpe’a	-0,044	Bliski zeru — brak przewagi
Najlepsza transakcja	+3,06 \$	Pojedynczy wynik odstający
Najgorsza transakcja	-3,72 \$	Pojedynczy wynik odstający

Tabela 5. Podsumowanie statystyczne wyników.

Szczególnie wymowna jest obserwacja dotycząca wyników odstających: po usunięciu jednej najlepszej transakcji (+3,06 USD) łączny wynik rzeczywisty spada z -2,30 USD do **-5,36 USD**. Oznacza to, że dodatnie rezultaty pojedynczych dni opierały się na nielicznych, wyjątkowych transakcjach, a nie na powtarzalnej skuteczności — co jest cechą charakterystyczną dla wyników losowych, a nie strategii posiadającej rzeczywistą przewagę.

9. Wnioski

9.1. Wnioski inżynierskie

Od strony inżynierskiej projekt należy uznać za udany. Zbudowano kompletny, wielomodułowy system, który:

- integruje pięć niezależnych usług zewnętrznych (broker, model językowy, dwa źródła newsów, Telegram),
- działa autonomicznie w trybie ciągłym, z obsługą błędów i automatycznym wznawianiem sesji,
- zapewnia trwałość stanu między cyklami i po restarcie,
- rejestruje dane, mierzy koszty transakcyjne i generuje czytelne raporty,
- został wdrożony jako usługa systemowa (systemd) na serwerze Linux.

Projekt pozwolił na praktyczne opanowanie zagadnień takich jak: praca z REST API i uwierzytelnianiem, przetwarzanie danych w pandas, harmonogramowanie zadań, obsługa wyjątków, persystencja danych w formacie JSON oraz generowanie dokumentów Excel.

9.2. Wnioski badawcze

Analiza danych jednoznacznie wykazała, że zaimplementowana strategia **nie posiada statystycznie istotnej przewagi rynkowej**. Wynik teoretyczny bliski zeru ($-0,82$ USD) oraz wskaźnik typu Sharpe’a na poziomie $-0,044$ wskazują, że decyzje systemu są zbliżone do losowych. Co istotne, problemem nie są koszty transakcyjne — okazały się one niewielkie (łącznie $1,48$ USD spreadów) — lecz **sam sygnał decyzyjny**, oparty na wskaźnikach technicznych z interwału 15-minutowego ocenianych przez model językowy.

Wniosek ten jest zgodny z szeroko opisaną w literaturze trudnością uzyskania trwałej przewagi w handlu krótkoterminowym (intraday) na podstawie ogólnodostępnych danych. Należy go traktować jako wartościowy rezultat: zbudowany system pomiarowy umożliwił obiektywną, ilościową weryfikację hipotezy — co jest istotą podejścia inżyniersko-naukowego.

9.3. Możliwe kierunki dalszego rozwoju

8. Zmiana horyzontu czasowego z 15 minut na dłuższy (np. dzienny lub tygodniowy), gdzie sygnał techniczny zawiera mniej szumu.
9. Wprowadzenie testów historycznych (backtestingu) na danych z 1–2 lat przed wdrożeniem jakiegokolwiek strategii na żywo.
10. Rezygnacja z modelu językowego jako bezpośredniego komponentu decyzyjnego na rzecz precyzyjnie zdefiniowanych, testowalnych reguł.
11. Porównanie wyników strategii z prostą strategią odniesienia (np. „kup i trzymaj” indeks), aby ocenić jej realną wartość dodaną.
12. Rozszerzenie zestawu testów jednostkowych dla modułów obliczeniowych (indicators, trade_logger).

10. Podsumowanie

Celem projektu było zaprojektowanie i implementacja kompletnego systemu algorytmicznego handlu w języku Python — i cel ten został osiągnięty. Powstała w pełni działająca, wielomodułowa aplikacja, która w sposób autonomiczny analizuje rynek, podejmuje decyzje i raportuje wyniki, integrując przy tym liczne zewnętrzne usługi.

Równie ważnym rezultatem jest przeprowadzona w oparciu o rzeczywiste dane analiza skuteczności, która wykazała brak statystycznej przewagi przyjętej strategii. Świadoma i uczciwa interpretacja tego wyniku — wraz z identyfikacją jego przyczyn i propozycją dalszych kierunków — stanowi, zdaniem autorów, najcenniejszy element całego przedsięwzięcia, łączący kompetencje programistyczne z myśleniem analitycznym.

Projekt pokazał, że zbudowanie technicznie poprawnego systemu i osiągnięcie nim zysku to dwa odrębne wyzwania — i że rzetelny pomiar potrafi dać odpowiedź na pytanie, którego sama implementacja nie rozstrzyga.